Acceso Remoto Seguro al Servidor de la U.E. Horizonte: Evidencia de TELNET frente a SSH, SSH Endurecido con fail2ban y DNS con Vistas y DNSSEC

Informe Técnico de la Actividad 03 (Laboratorios 9, 10 y 11)

Grupo Sin Hosting:

Jacqueline Cáceres, Lucas Carrasco, José María Orihuela,

Juniors Parapaino, Limber Quispe y Lizeth Rodríguez

Carrera de Ingeniería de Sistemas, Universidad Privada Domingo Savio

SIS-0150: Tecnología Web I

M.Sc. Ing. Jimmy Nataniel Requena Llorentty

25 de octubre de 2026

Repositorio: https://github.com/Grupo-Sin-Hosting/Actividad-03-SSH

Acceso Remoto Seguro al Servidor de la U.E. Horizonte: Evidencia de TELNET frente a SSH, SSH Endurecido con fail2ban y DNS con Vistas y DNSSEC

El técnico anterior de la U.E. Horizonte administraba el servidor del colegio con TELNET "porque era más simple". El servidor pronto saldrá a internet con su dominio público, y un consultor advirtió además que desde fuera se veían los nombres internos y que el dominio podía ser suplantado. Esta actividad responde a ese escenario. Primero se demuestra con capturas reales la diferencia de seguridad entre TELNET y SSH. Después se deja el servidor accesible **solo** por SSH endurecido, con fail2ban contra la fuerza bruta. Como base, se protege el DNS de la organización con vistas y DNSSEC.

El trabajo se hizo en contenedores Docker con Ubuntu 24.04: el servidor del colegio de los laboratorios 2 al 5 (`srv-horizonte`) y la red de DNS de la Actividad 2, ampliada con una red que simula internet. Cada resultado se comprobó con capturas propias y con los verificadores de la cátedra: 16/16 (Laboratorio 9), 11/11 (Laboratorio 10) y 15/15 (Laboratorio 11).

Objetivos

Objetivo General

Demostrar con evidencia la diferencia de seguridad entre TELNET y SSH y dejar el servidor de la organización accesible solo por SSH endurecido.

Objetivos Específicos

- Capturar con Wireshark una sesión TELNET en la red aislada del laboratorio y analizar el tráfico en texto claro.
- Capturar la misma sesión por SSH y comparar el tráfico cifrado, incluida la identidad del servidor y el intercambio de claves.

- Configurar claves ed25519 y un `sshd_config` endurecido (sin root, sin contraseña, puerto y usuarios permitidos), con `scp`, `sftp` y un túnel local.
- Instalar fail2ban, simular un ataque de fuerza bruta y registrar el bloqueo.
- Separar lo que el colegio e internet ven del DNS (vistas) y firmar el dominio público con DNSSEC.
- Documentar el uso de inteligencia artificial en una bitácora verificada.

Arquitectura de la Solución

La Tabla 1 y la Figura 1 resumen el entorno. El servidor `srv-horizonte` es el que se endurece. Hacia fuera, Docker solo publica el SSH en el puerto **2222** del equipo anfitrión, junto con la web en 8080 y 8443 (Figura 2); el puerto 22 queda dentro de la red de contenedores. La red `red_dns` aloja el DNS con vistas. ns1 tiene una cara hacia el colegio (10.10.10.2) y otra hacia "internet" (172.30.0.2), donde está el nodo `externo`.

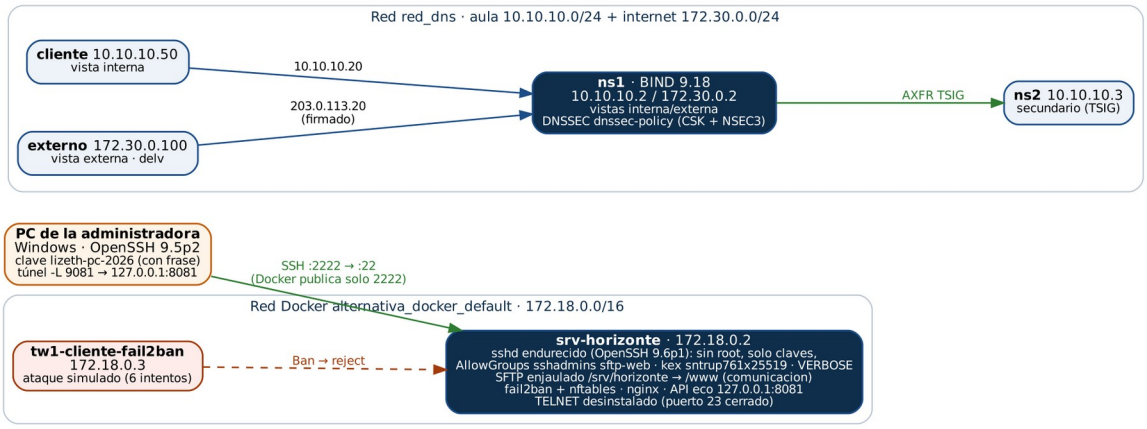
Tabla 1

Equipos del Laboratorio

Equipo	Dirección	Software	Función
`srv-horizonte`	172.18.0.2 (2222→22 en el anfitrión)	Ubuntu 24.04 · OpenSSH 9.6p1 · fail2ban · nginx	Servidor del colegio: SSH endurecido, SFTP, web y API interna
PC de la administradora	Windows (anfitrión)	OpenSSH for Windows 9.5p2 · Wireshark 4.4	Cliente SSH con clave propia, túnel y análisis de capturas
`tw1-cliente-fail2ban`	172.18.0.3 (temporal)	Ubuntu 24.04 (cliente ssh)	Atacante simulado de fuerza bruta
ns1	10.10.10.2 · 172.30.0.2	BIND 9.18	Autoritativo con vistas interna y externa; DNSSEC
ns2	10.10.10.3	BIND 9.18	Secundario de la vista interna (TSIG)
cliente / externo	10.10.10.50 / 172.30.0.100	dig, delv	Prueban las dos caras del DNS

Figura 1

Arquitectura de la solución



Nota. Elaborado por el grupo. La flecha discontinua roja es el ataque simulado, bloqueado por fail2ban.

Figura 2

El servidor solo publica SSH en el puerto 2222 del anfitrión



Laboratorio 9: DNS Profesional con Vistas y DNSSEC

Antes de las vistas, desde `externo` se obtenían las IP privadas del colegio: `www.horizonte.internal` → 10.10.10.20 y `mail` → 10.10.10.25 (Figura 3). Era reconocimiento gratis para un atacante. Las vistas de BIND 9.18 resuelven esto: ns1 elige la **primera** vista cuyo `match-clients` coincide con la IP de origen. La vista interna (`acl "red-`

`colegio`") va antes que la externa (`any`). Con vistas, todas las zonas deben estar dentro de una vista, por eso se quitó el `include` de `named.conf.default-zones`. El resultado es el mismo nombre con dos respuestas (Tabla 2, Figuras 4 y 5).

Figura 3

Antes de las vistas: desde internet se ven las IP internas

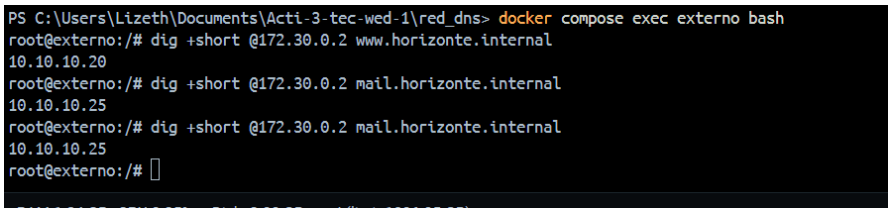


Tabla 2

El Mismo Nombre, Dos Respuestas

Consulta	Desde el colegio (10.10.10.50)	Desde internet (172.30.0.100)
`www.ue-horizonte.example`	`10.10.10.20`	`203.0.113.20`
`intranet.ue-horizonte.example`	`10.10.10.21`	`NXDOMAIN`
`www.horizonte.internal`	`10.10.10.20`	`REFUSED`

Figura 4

Vista interna desde el cliente del colegio

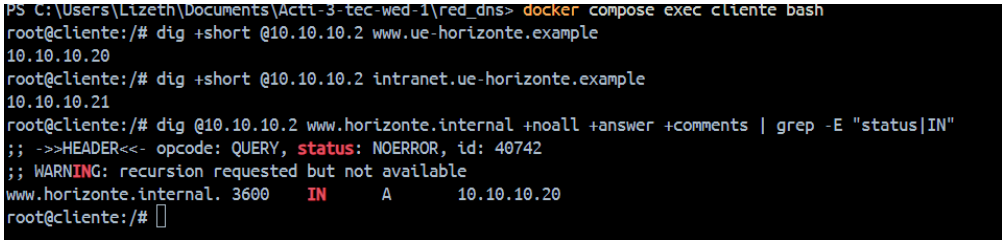
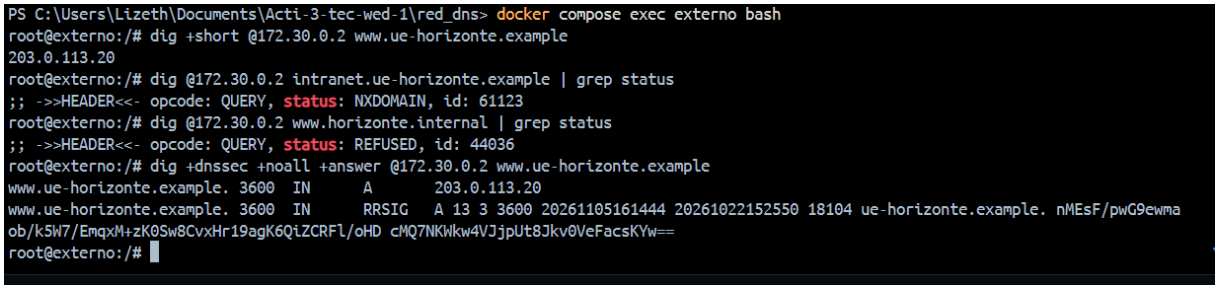


Figura 5

Vista externa: IP pública, NXDOMAIN, REFUSED y firma RRSIG



La zona pública se firma con `dnssec-policy "horizonte"`: una CSK ECDSA P-256 (algoritmo 13), firmas de 14 días renovadas cada 5 y **NSEC3 con 0 iteraciones y sin sal**, como recomienda el RFC 9276. En BIND 9.18 una zona estática exige `inline-signing yes`. `delv`, con la clave pública como ancla de confianza, respondió `fully validated`, tanto para `www` como para un nombre inexistente probado con NSEC3 (Figura 6). Al cambiar **una sola letra** del ancla, la respuesta fue `broken trust chain` (Figura 7): sin la clave privada nadie puede suplantar el dominio. El DS para el registrador quedó generado (`18104 13 2 0AD9…`) y `vigilar_dns.sh` vigila que las firmas no venzan. El verificador dio 16 de 16 (Figura 8).

Figura 6

delv: respuesta y NXDOMAIN validados con el ancla

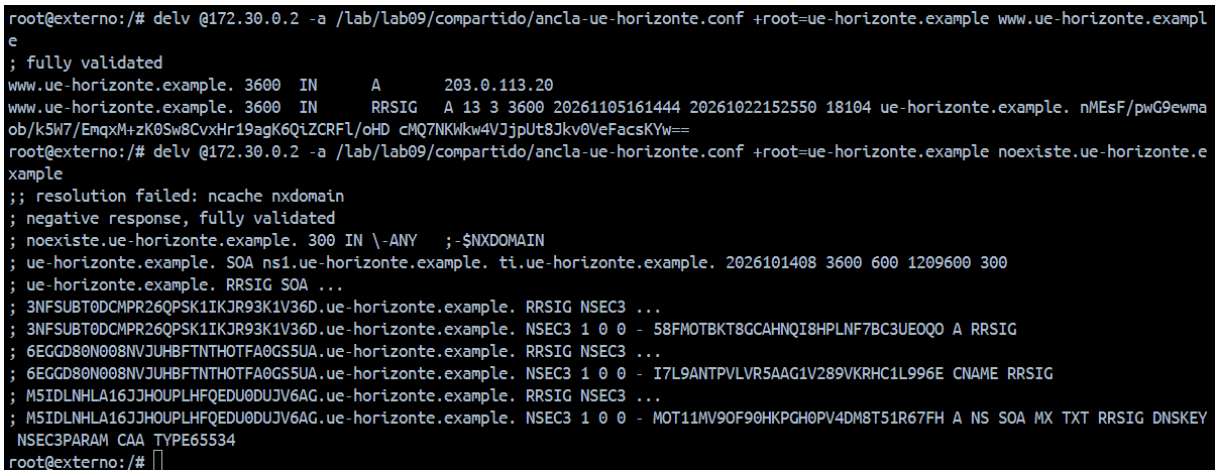


Figura 7

Ancla con una letra cambiada: broken trust chain



Figura 8

verificar_lab09.sh: 16 de 16 comprobaciones



Laboratorio 10: Evidencia de TELNET frente a SSH

La Prueba del Espía

En la red aislada del laboratorio (la interfaz de bucle local del servidor) se activó TELNET solo para la demostración. Ubuntu lo instala desactivado y fue necesario habilitarlo en `/etc/inetd.conf`. `capturar.sh` abrió una sesión con la usuaria de prueba `secretaria`, escribió un texto confidencial y la cerró mientras `tcpdump` grababa. Después hizo lo mismo por SSH (Figura 9). Las capturas se abrieron en **Wireshark** con *Analizar* > *Seguir* > *Flujo TCP*. En TELNET se leen `login:`, `secretaria`, `Password:` y `Circular2026` tal cual (Figura 10). En

SSH solo se leen las versiones y la lista de algoritmos; todo lo demás está cifrado, y buscar la contraseña no da resultados (Figura 11). La Tabla 3 resume la comparación.

Figura 9

capturar.sh: lo que se lee en claro en TELNET y no en SSH

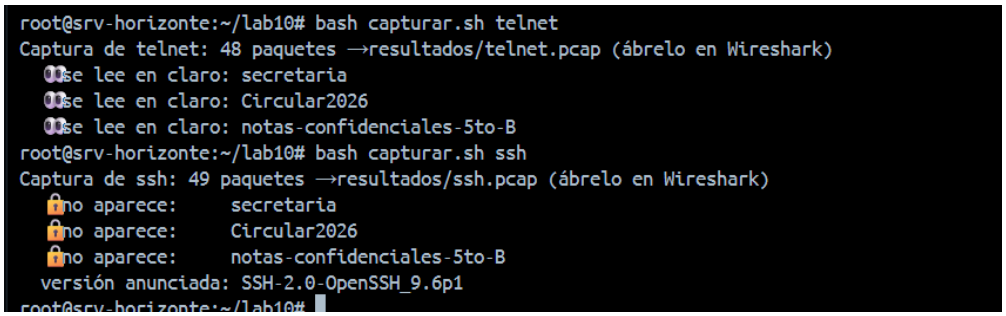


Figura 10

Wireshark, flujo TCP de TELNET: usuario y contraseña en texto plano

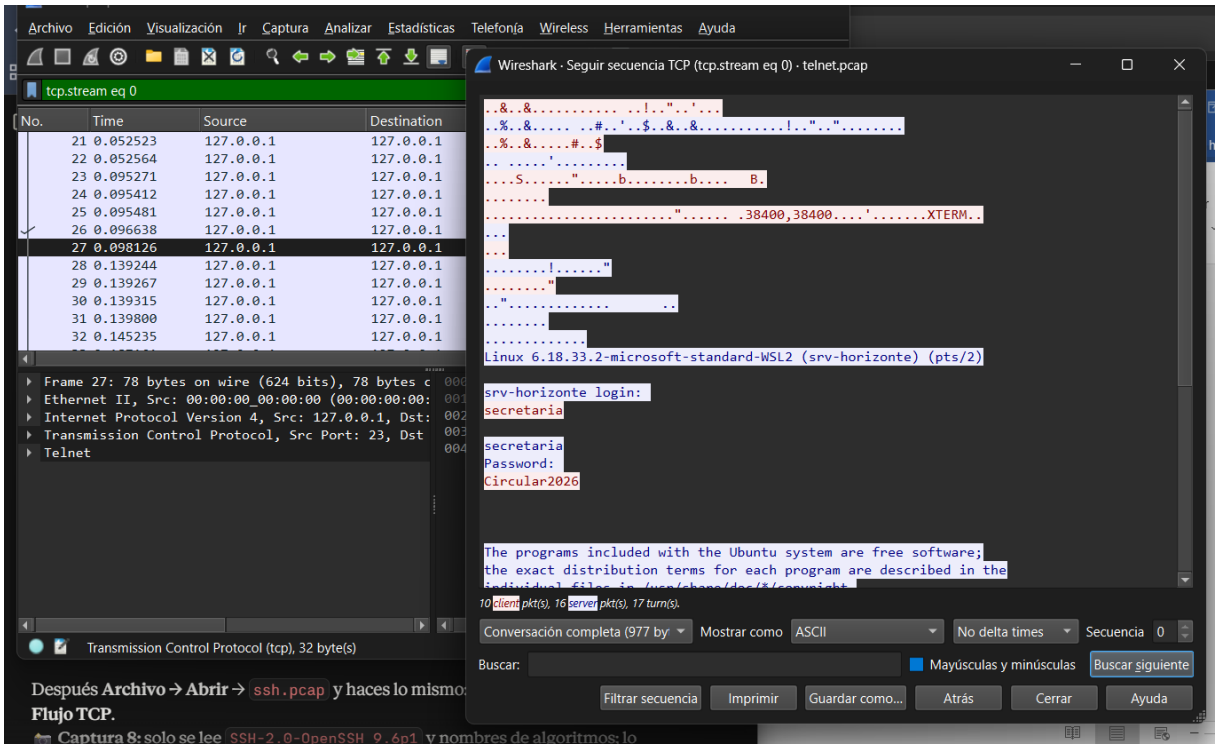


Figura 11

Wireshark, flujo TCP de SSH: solo versión y algoritmos; el resto, cifrado

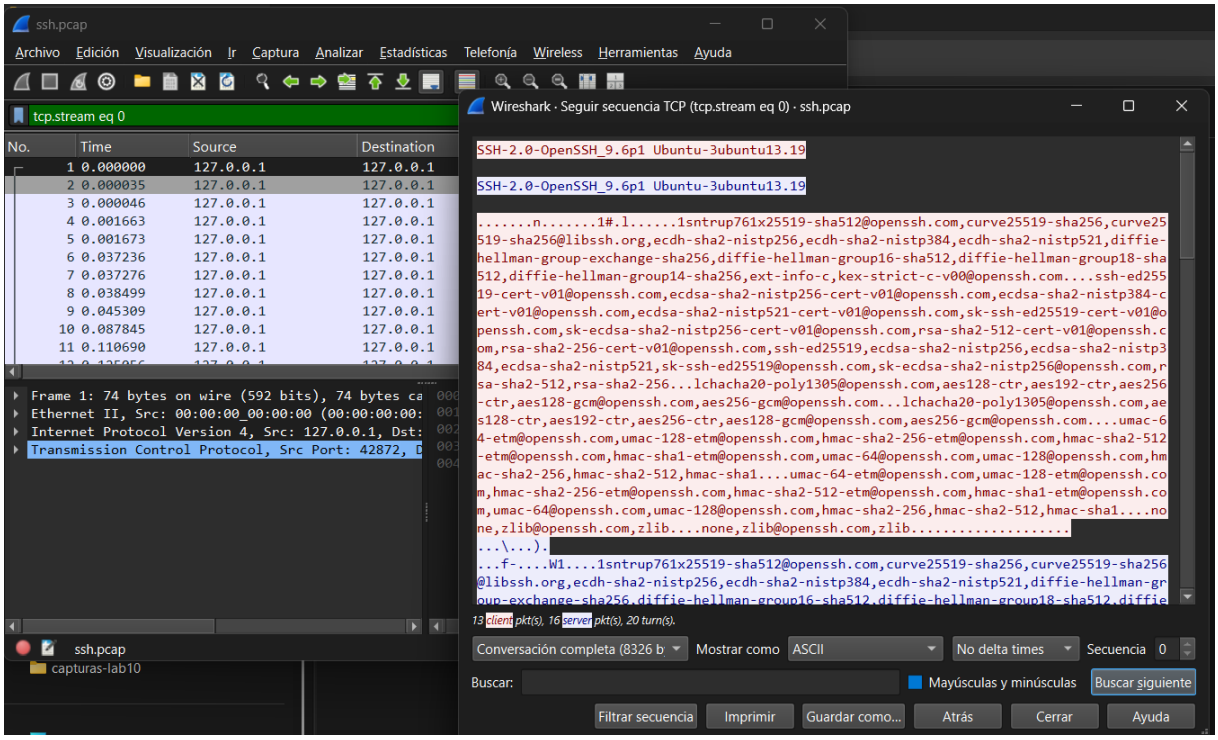


Tabla 3

Qué Ve un Espía en la Red

	TELNET (puerto 23)	**SSH (puerto 22)**
Usuario	**Visible**: `secretaria`	No visible
Contraseña	**Visible**: `Circular2026`	No visible
Lo escrito en la sesión	**Visible**: `notas-confidenciales-5to-B`	No visible
Lo que se lee en claro	Todo: banner, mensajes, comandos y respuestas	Solo `SSH-2.0-OpenSSH_9.6p1` y los nombres de algoritmos negociados

Nota. La contraseña es de una usuaria creada solo para el laboratorio. Al terminar, TELNET se desactivó y se desinstaló (puerto 23 cerrado).

Identidad del Servidor y Acceso con Clave

La clave de host traía el comentario `root@buildkitsandbox`: se generó al construir la imagen y la comparten todos los que la usan. Así, cualquiera con esa imagen podría hacerse

pasar por el servidor de otro (ataque de intermediario). Se regeneró con `ssh-keygen -A` y quedó la huella `SHA256:KueRdm7E…` `root@srv-horizonte`. `ssh -v` mostró que se negocia **`sntrup761x25519-sha512`**, un intercambio híbrido post-cuántico (NTRU Prime + X25519), con cifrado ChaCha20-Poly1305 (Figura 12). La administradora `lizeth` recibió una clave **ed25519** (`-a 64`): la privada nunca sale del cliente y la pública va a `authorized_keys` con permisos 600, dentro de `.ssh` con 700. El alias `horizonte` de `~/.ssh/config` permite entrar sin contraseña (Figura 13). Con `scp` se publicó `comunicado.html`, que quedó de `lizeth` y del grupo `webadmin` por el bit setgid del sitio, y también se probaron `sftp` y `rsync`. El verificador dio 11 de 11 (Figura 14).

Figura 12

Algoritmos negociados: kex híbrido post-cuántico

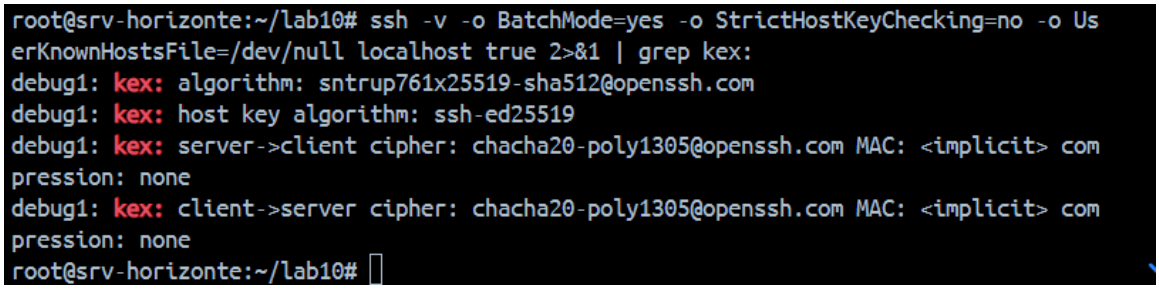


Figura 13

Claves de host regeneradas y acceso con clave ed25519

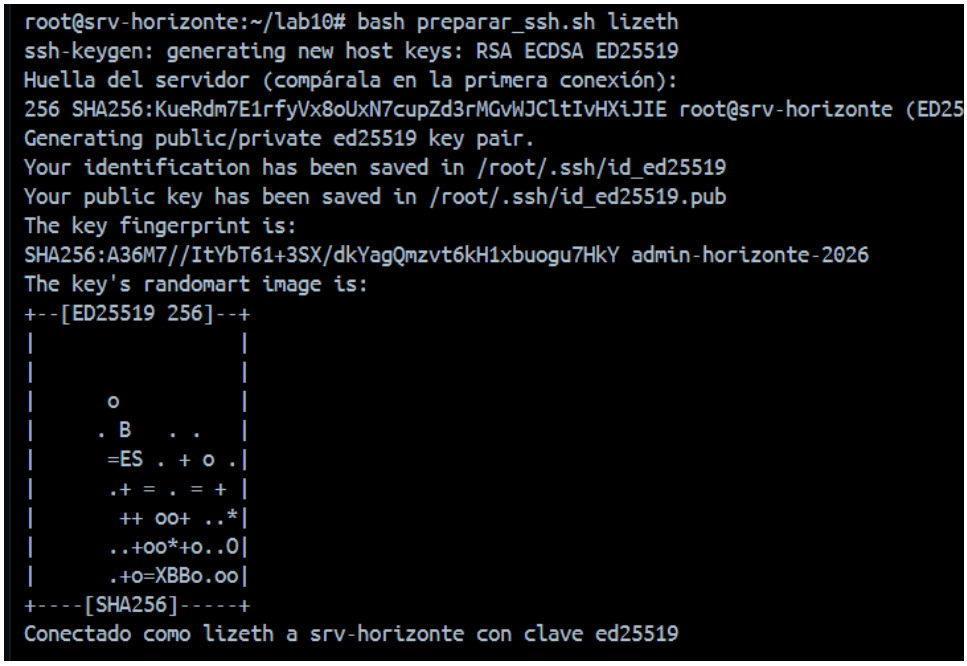
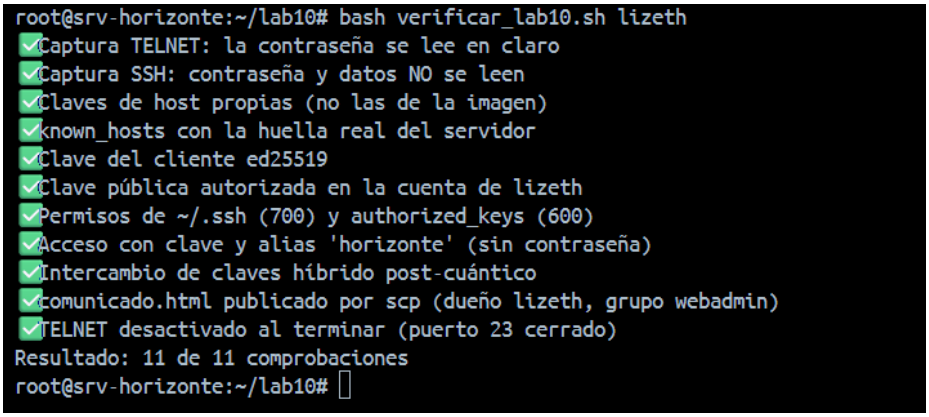


Figura 14

verificar_lab10.sh: 11 de 11 comprobaciones



Laboratorio 11: Servidor Accesible Solo por SSH Endurecido

sshd_config Endurecido

La configuración va en `/etc/ssh/sshd_config.d/10-horizonte.conf`. Como el `sshd_config` de Ubuntu la incluye al principio y en sshd gana el primer valor leído, manda

sobre los valores del paquete sin editarlo. Se cambió con dos sesiones abiertas y validando con `sshd -t` antes de reiniciar. La Tabla 4 compara el antes (Figura 15) y el después (Figura 16). Las pruebas de la Figura 17 muestran que `lizeth` entra con su clave y ve el aviso legal. root y cualquier intento sin clave reciben `Permission denied (publickey)`: el servidor ya ni ofrece contraseña.

Figura 15

Auditoría inicial con sshd -T

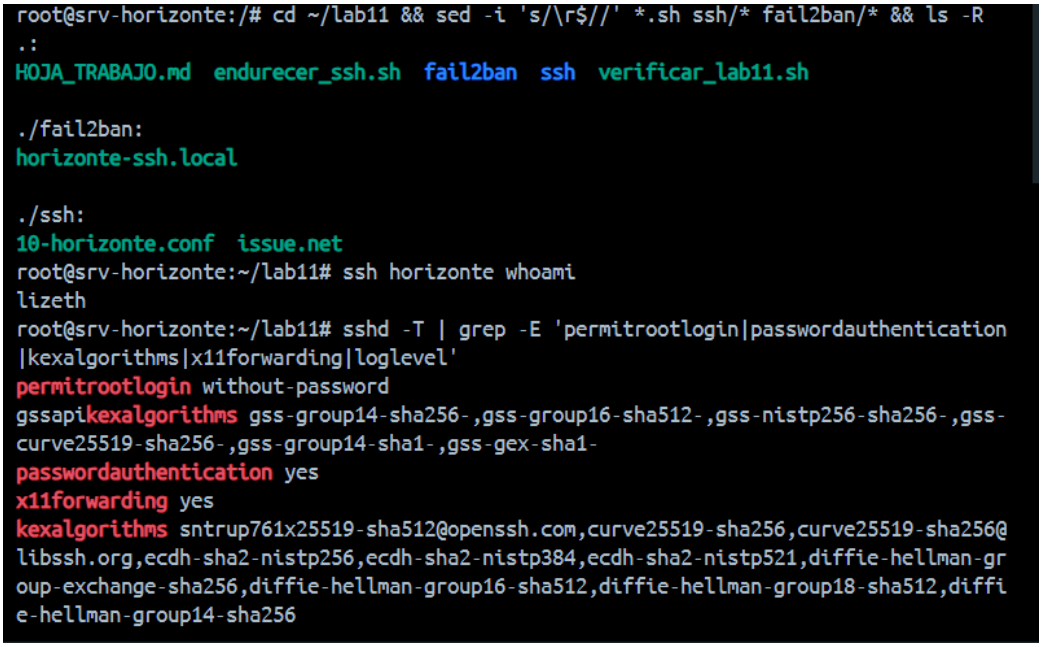


Tabla 4

sshd Antes y Después del Endurecimiento

Requisito	Antes	Después
Sin root	`PermitRootLogin without-password`	`PermitRootLogin no`
Sin contraseña	`PasswordAuthentication yes`	`PasswordAuthentication no` + `AuthenticationMethods publickey`
Usuarios permitidos	Cualquier cuenta	`AllowGroups sshadmins sftp-web` (lizeth en `sshadmins`; `comunicacion` en `sftp-web`)
Puerto	Publicado en el anfitrión como 2222 → 22	Se mantiene: solo 2222 hacia fuera; 22 dentro de la red de contenedores, cubierto por fail2ban

Requisito	Antes	Después
Criptografía	10 algoritmos de kex	Solo `sntrup761x25519-sha512`, `curve25519-sha256` (×2)
Registro	`LogLevel INFO`	`LogLevel VERBOSE`: huella de la clave en cada acceso
Otros	`X11Forwarding yes`	`X11Forwarding no`, `MaxAuthTries 3`, `LoginGraceTime 20`, aviso legal ASCII

Nota. Sobre el puerto: el grupo decidió no mover sshd del 22 dentro del contenedor y publicarlo hacia fuera solo en 2222. Cambiar el puerto reduce el ruido de los bots, pero un escaneo lo encuentra en segundos. La protección real son las claves, AllowGroups y fail2ban.

Figura 16

Después de endurecer: jaula root:root y sshd -T con los valores nuevos

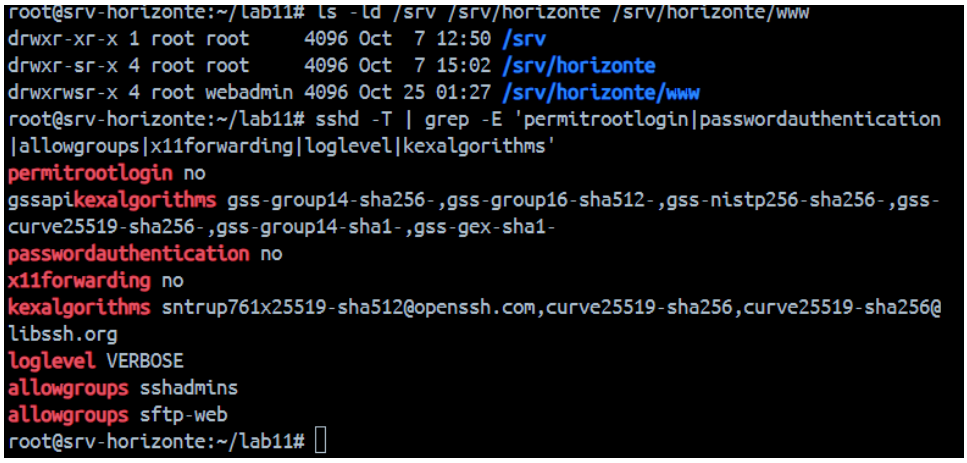
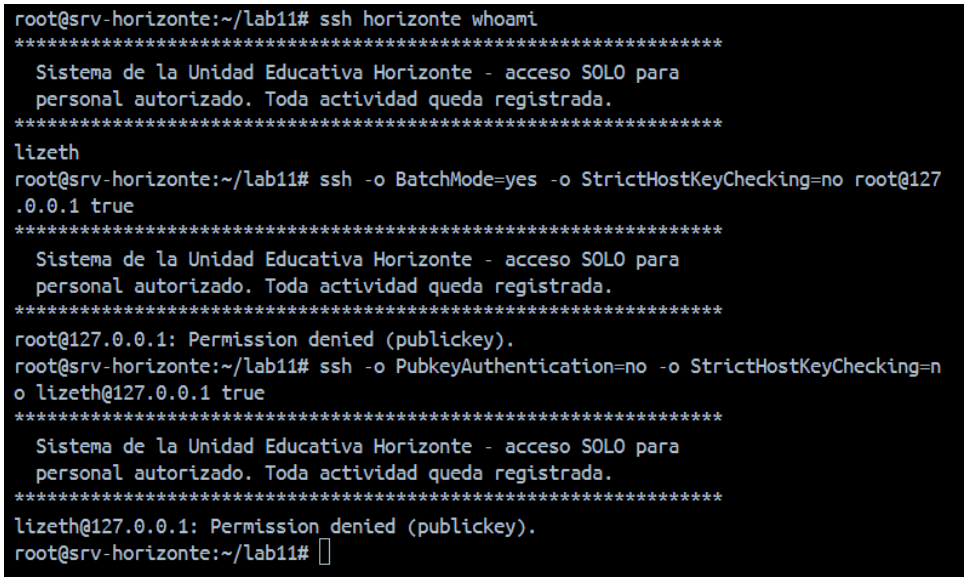


Figura 17

lizeth entra; root y el acceso sin clave son rechazados



scp, SFTP Enjaulado y Túnel Local

La encargada de comunicación publica sin conocer ninguna contraseña de administrador. Su cuenta `comunicacion` (grupo `sftp-web`) solo admite SFTP, está encerrada en `/srv/horizonte` y empieza en `/www`. Sus archivos quedan con 664 y grupo `webadmin`, `cd /etc` no existe para ella y pedir una terminal devuelve `This service allows sftp connections only.` (Figura 18). La jaula exige que `/srv/horizonte` y sus carpetas superiores sean de root y no escribibles por el grupo. Para demostrarlo, se le devolvió la escritura al grupo un momento y sshd cortó la sesión con `bad ownership or modes for chroot directory` (Figura 19).

Figura 18

comunicacion publica por SFTP y no puede salir de la jaula

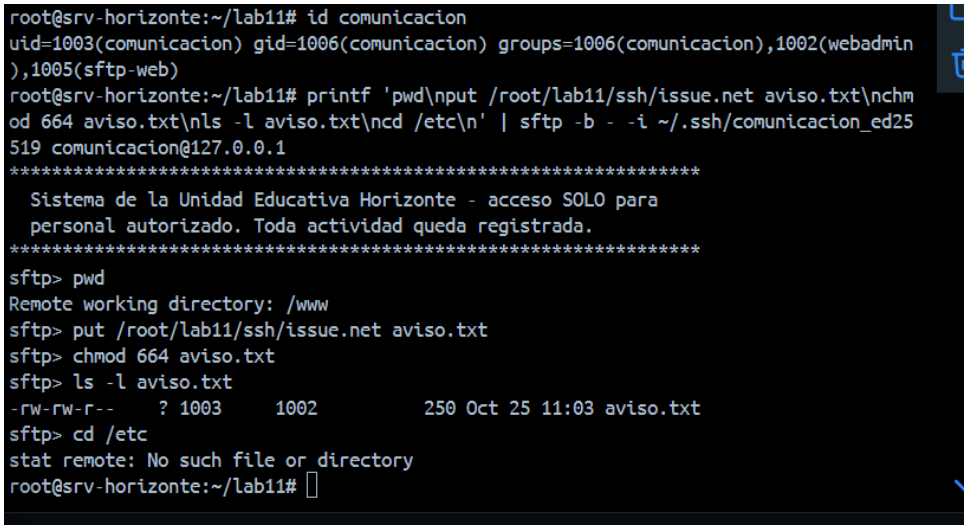
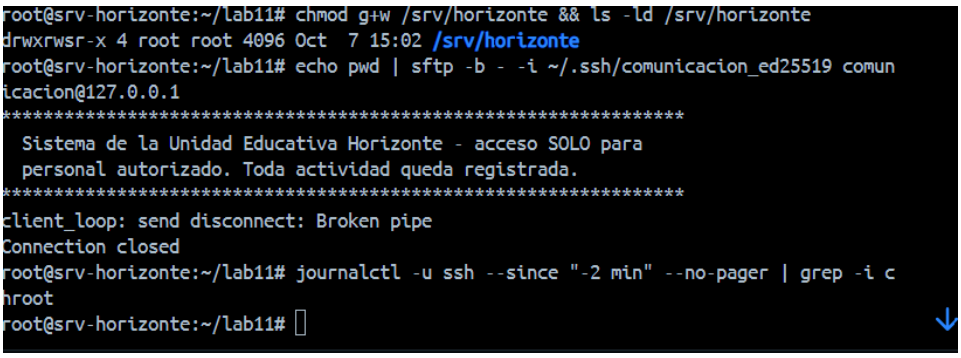


Figura 19

Con la jaula escribible por el grupo, sshd corta la sesión



fail2ban: Ataque Simulado y Bloqueo

fail2ban vigila el journal de **`ssh.service`**, nombre de la unidad en Ubuntu 24.04. El filtro por defecto espera `sshd.service`, por eso hubo que fijar `journalmatch`. La regla es 4 intentos en 10 minutos: bloqueo de 1 hora, que crece si la IP reincide. Al probarlo apareció un problema que no estaba en la guía: los mensajes de sshd no llegaban al journal. `/dev/log` pertenecía a `/usr/sbin/syslogd` (Figura 20), del paquete `inetutils-syslogd`, que se instaló el 23/10 como dependencia de TELNET en el Laboratorio 10 y quedó al desinstalarlo (Figura

21). Sin journal, fail2ban habría quedado ciego. Se purgó el paquete, junto con el demonio telnetd sobrante, y se restauró el enlace a journald.

Figura 20

/dev/log pertenecía a syslogd y no a journald

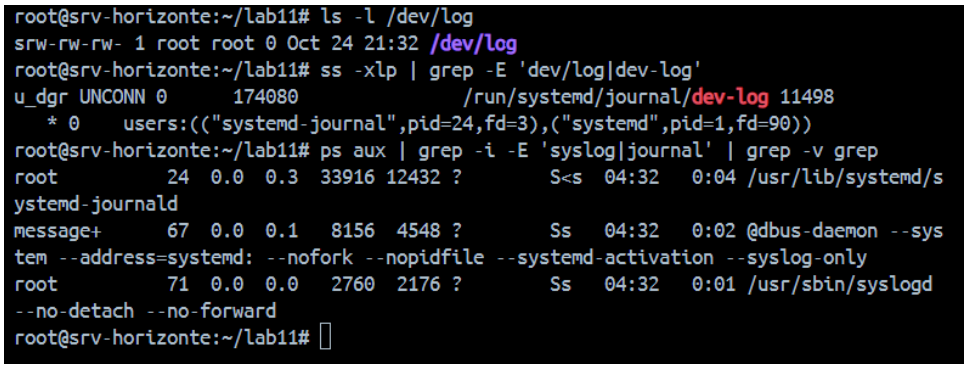
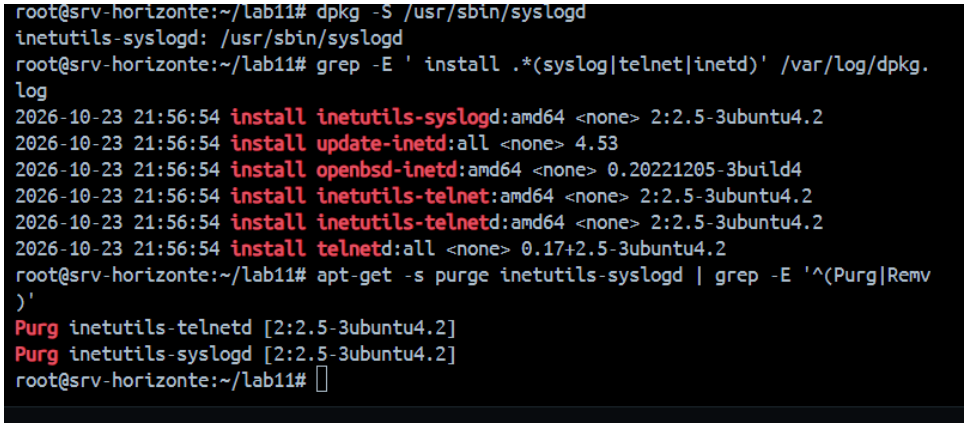


Figura 21

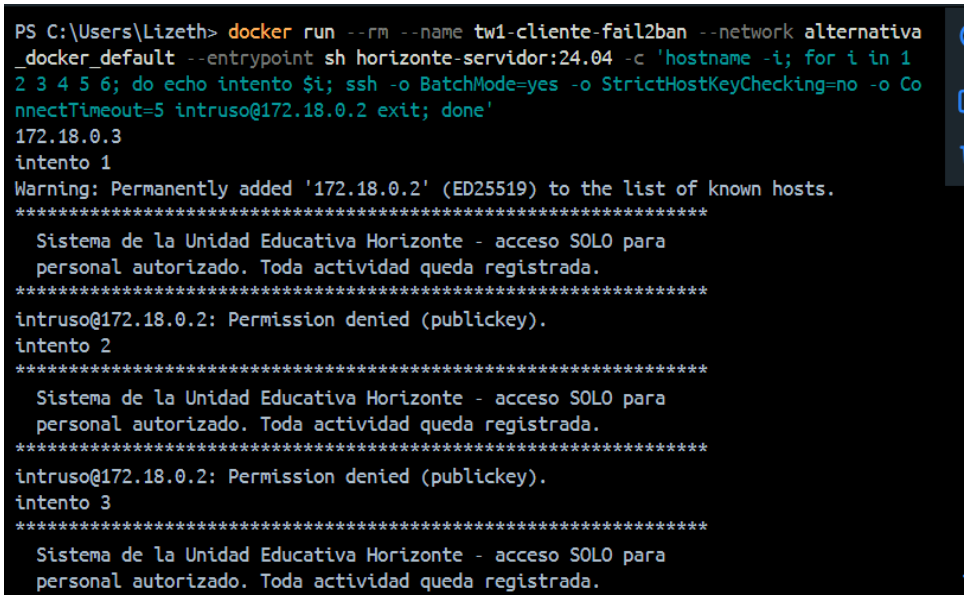
inetutils-syslogd, instalado junto con TELNET en el Laboratorio 10



El ataque se simuló desde un contenedor temporal en la misma red (172.18.0.3), con 6 intentos de un usuario inexistente y solo contra el servidor del curso. Los tres primeros recibieron `Permission denied`. Después fail2ban registró el `Ban` y nftables empezó a rechazar la IP, así que los siguientes vieron `Connection refused` (Figura 22, Tabla 5). La IP quedó en el conjunto `addr-set-sshd` y el registro muestra la secuencia `Ban 11:30:25 → Unban 11:46:10 → Ban 11:47:03` (Figura 23). Al terminar se desbloqueó.

Figura 22

Ataque simulado: Permission denied antes del bloqueo



Nota. La captura recorta la salida en el tercer intento; los intentos 4 a 6 de esa ejecución respondieron `Connection refused`.

Tabla 5

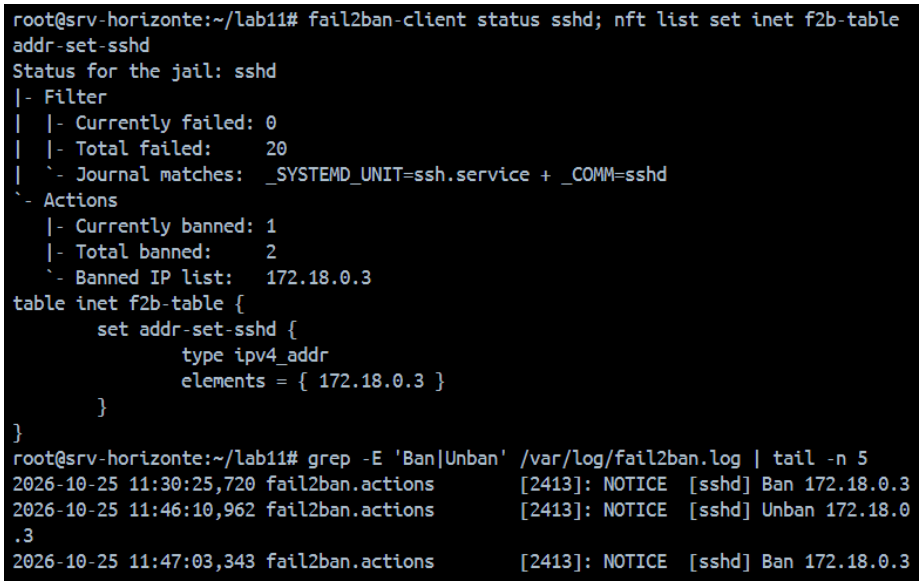
Respuesta del Servidor Durante la Fuerza Bruta

Intento	Lo que ve el atacante	Lo que pasa en el servidor
1 a 3	`Permission denied (publickey)`	sshd: `Invalid user intruso from 172.18.0.3`; fail2ban cuenta cada línea (`Found`)
4 a 6	`Connection refused`	`Ban 172.18.0.3`; nftables: `tcp dport 22 ip saddr @addr-set-sshd reject`

Nota. En modo `aggressive` cada conexión genera más de una falla, por eso el bloqueo llegó en menos de 2 segundos. Totales: 20 fallas detectadas y 2 bloqueos.

Figura 23

fail2ban y nftables: IP bloqueada y registro de Ban/Unban



Por último, el túnel. La PC con Windows usó su propia clave (`lizeth-pc-2026`, con frase). Al conectarse apareció `REMOTE HOST IDENTIFICATION HAS CHANGED`, porque la PC recordaba la clave vieja de la imagen. Se verificó la huella **por otro canal** (`docker exec … ssh-keygen -lf` → `KueRdm7E…`) y solo entonces se borró la entrada antigua (Figura 24). Las primeras conexiones las cortó `LoginGraceTime 20` mientras se comparaba la huella. Con `ssh -N -L 9081:127.0.0.1:8081`, la API interna del Laboratorio 4, que solo escucha en el servidor, respondió en la PC con `"cliente": "127.0.0.1"` (Figura 25). El verificador dio 15 de 15 (Figura 26).

Figura 24

Huella verificada, entrada vieja borrada y clave de la PC autorizada

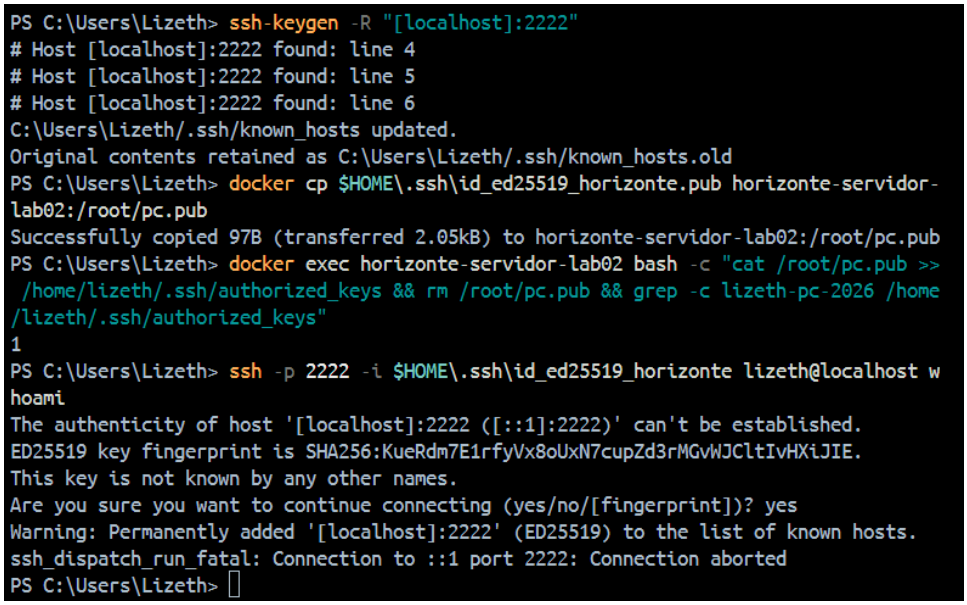


Figura 25

La API interna alcanzada desde la PC por el túnel SSH



Figura 26

verificar_lab11.sh: 15 de 15 comprobaciones



Problemas Encontrados y Soluciones

Ningún laboratorio salió a la primera. La Tabla 6 resume los problemas reales y cómo se resolvieron. En todos los casos se confirmó la causa con un comando antes de cambiar la configuración.

Tabla 6

Problemas Encontrados Durante la Actividad

Lab	Síntoma	Causa	Solución
9	`when using 'view' statements, all zones must be in views`	El `include` de `default-zones` seguía; VS Code no había guardado	Borrar la línea y comprobar con `grep` en ns1
9	Monitoreo: `ns1=2026100701 ns2=2026100703`	El serial nuevo era menor que el que tenía ns2	Subir a 2026102201 y `rndc reload … in interna`
10	Error de sintaxis en `preparar_ssh.sh`	Al pegar se borraron `EOF` y `fi`	Reemplazar el archivo completo antes de ejecutar
10	`comunicado.html` con `-rwxr-xr-x`	`docker cp` desde Windows dejó 755 y `scp` lo conserva	`chmod 664` y `scp -p`
11	fail2ban no veía nada en el journal	`inetutils-syslogd` (traído por TELNET) se apropió de `/dev/log`	Simular y purgar; enlace `/dev/log` → journald
11	`REMOTE HOST IDENTIFICATION HAS CHANGED` en Windows	Huella vieja de la imagen, anterior al Lab 10	Verificar por otro canal y `ssh-keygen -R` de esa entrada

Lab	Síntoma	Causa	Solución
11	`Connection aborted` desde Windows	`LoginGraceTime 20` venció durante la pregunta de la huella	Guardar la huella y reconectar

Resultados

Tabla 7

Resultados de los Verificadores

Laboratorio	Script	Resultado	Qué valida
9	`verificar_lab09.sh` (en ns1)	**16 de 16**	Vistas, DNSSEC con NSEC3, delv válido y con ancla falsa, DS y monitoreo
10	`verificar_lab10.sh lizeth`	**11 de 11**	Capturas TELNET/SSH, claves de host propias, ed25519, permisos, kex post-cuántico, scp, TELNET cerrado
11	`verificar_lab11.sh lizeth`	**15 de 15**	Sin root, solo claves, AllowGroups, SFTP enjaulado, huella en el registro, fail2ban con nftables y túnel

Repositorio en GitHub

Los archivos completados, las hojas de trabajo, las bitácoras, este informe y la presentación están en el repositorio del grupo: https://github.com/Grupo-Sin-Hosting/Actividad-03-SSH. La Tabla 8 muestra su estructura. El archivo `.gitignore` excluye claves privadas SSH, archivos `.private` y `.key` de DNSSEC y TSIG, y la carpeta `lab07/compartido/`. Las capturas `.pcap` del Laboratorio 10 sí se incluyen porque la guía las pide, y solo contienen la contraseña de prueba del laboratorio.

Tabla 8

Estructura del Repositorio

Carpeta o archivo	Contenido
`red_dns/`	`compose.yaml` con la red `internet` y el nodo `externo`, y el `Dockerfile`
`lab07/`	Instaladores de ns1 y ns2 que el Laboratorio 9 reutiliza (sin la clave TSIG)
`lab09/`	`ns1/` (vistas y `dnssec-policy`), `vigilar_dns.sh`, `compartido/` (DS y ancla públicos), hoja y bitácora
`lab10/`	`preparar_ssh.sh`, `resultados/` (`telnet.pcap`, `ssh.pcap` y resúmenes), hoja y bitácora

Carpeta o archivo	Contenido
`lab11/`	`ssh/10-horizonte.conf`, `fail2ban/horizonte-ssh.local`, `endurecer_ssh.sh`, hoja y bitácora
`informe/`	Este informe (`.docx` y `.pdf`) y los informes de cada laboratorio
`presentacion/`	Presentación de la actividad en `.pptx`
`ROLES.md`	Distribución del trabajo y de la defensa

Distribución del Trabajo y de la Defensa

Cada integrante es responsable de una parte de la defensa, con su rama e issue en el repositorio (Tabla 9).

Tabla 9

Roles del Grupo

Integrante	Parte	Debe poder explicar
Lucas Carrasco	Entorno y DNS profesional (Lab 9)	Arquitectura, vistas y su orden, DNSSEC con `dnssec-policy`, delv con ancla falsa, DS
Jacqueline Cáceres	TELNET en claro (Lab 10, Parte A)	Captura con Wireshark, qué se lee en claro y por qué TELNET sigue en routers e impresoras
José María Orihuela	SSH cifrado e identidad (Lab 10, Partes B y C)	Comparación del tráfico, `buildkitsandbox`, kex post-cuántico, aviso de identidad cambiada
Lizeth Rodríguez	Claves ed25519 y sshd endurecido (Lab 11, Partes A y B)	Tabla antes/después, `sshd_config.d`, puerto, `AllowGroups`, `VERBOSE`; coordinación y repositorio
Juniors Parapaino	scp, SFTP enjaulado y túnel (Labs 10 D y 11 C, E)	Setgid de `webadmin`, regla del chroot, `ForceCommand`, túnel `-L`, `-D` y `-J`
Limber Quispe	fail2ban y fuerza bruta (Lab 11, Parte D)	`journalmatch`, el problema de `/dev/log`, el ataque, Ban/Unban y nftables

Bitácora de Uso de Inteligencia Artificial

Se usó un asistente de IA (Claude, de Anthropic) para entender opciones de configuración, interpretar salidas y formular hipótesis de diagnóstico. Se respetaron las reglas de las guías. No se pegaron claves privadas ni archivos `.private`. No se aceptaron huellas sin verificarlas. TELNET no quedó activo. El ataque fue solo contra el servidor del curso y sin diccionarios. Nunca se desactivó una protección "para que funcione". La Tabla 10 resume las consultas más importantes; las bitácoras completas de cada laboratorio están en el repositorio.

Tabla 10

Bitácora Consolidada de Uso de IA

Lab	Consulta	Respuesta de la IA	Cómo se verificó	Qué se aprendió
9	Orden de las vistas en BIND 9.18	Gana la primera vista que coincide; any al final	Cliente → 10.10.10.20; externo → 203.0.113.20	El orden es parte de la seguridad
9	Qué exige `dnssec-policy` sin `inline-signing`	Una zona estática necesita firma en línea	`named-checkconf` sobre una copia en `/tmp`	Probar errores en copias
9	Por qué ns2 no copiaba la zona	El serial del primario era menor	Serial 2026102201 en ambos; 16/16	El serial solo puede subir
10	Qué significa `buildkitsandbox`	Clave de host compartida por la imagen	Huella nueva `root@srv-horizonte`	Clonar también clona la identidad
10	Algoritmos de `ssh -v`	kex híbrido post-cuántico y cifrado autenticado	Coincide con Wireshark y el verificador	Riesgo de "guardar ahora, descifrar después"
11	Regla del `ChrootDirectory`	La raíz de la jaula y sus superiores, solo de root	Error `bad ownership` provocado y corregido	Por qué la jaula no puede ser escribible
11	Por qué el journal no registraba sshd	Revisar quién escucha en `/dev/log`	`ss -xlp`, `dpkg -S`, `dpkg.log`	Una dependencia sobrante puede romper otro servicio
11	Cómo probar fail2ban sin atacar a terceros	Contenedor temporal en la misma red	Ban en el log y la IP en nftables	Probar la protección, no suponerla
11	Aviso de identidad cambiada en Windows	Verificar por otro canal antes de `ssh-keygen -R`	Huellas iguales: `KueRdm7E…`	Aplicar la reflexión del Lab 10
Act. 3	Borrador del informe, diagrama y presentación	Estructura y redacción a partir de las capturas	Cada dato revisado contra las salidas reales	La IA redacta; el grupo verifica y defiende

Conclusiones

La evidencia es concluyente. Con TELNET, cualquiera que capture la red lee el usuario, la contraseña y todo lo escrito en la sesión. Con SSH solo se ven la versión y los algoritmos, y el intercambio de claves negociado incluso resiste a futuras computadoras cuánticas. El servidor del colegio quedó accesible solo por SSH endurecido: sin root, sin contraseñas, solo dos grupos, criptografía moderna y registro de la clave usada en cada acceso. Hacia fuera, solo el puerto 2222. La encargada de comunicación publica por SFTP encerrada en su carpeta. La API interna solo se alcanza por túnel, y quien insiste con intentos fallidos queda bloqueado en nftables en menos de dos segundos.

El DNS de la organización dejó de revelar los nombres internos y su dominio público está firmado: una sola letra cambiada en el ancla rompe la cadena de confianza. La lección más importante salió de un problema no previsto. Un paquete instalado para la demostración de TELNET dejó sin registros a fail2ban, y se descubrió solo porque el bloqueo se **probó** con un ataque real y controlado. Las medidas de seguridad hay que verificarlas, no suponerlas.

Referencias

Arends, R., Austein, R., Larson, M., Massey, D., & Rose, S. (2005). *DNS security introduction and requirements* (RFC 4033). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc4033

Fail2ban. (s.f.). *Fail2ban* [Software y documentación]. GitHub. https://github.com/fail2ban/fail2ban

Hardaker, W., & Dukhovni, V. (2022). *Guidance for NSEC3 parameter settings* (RFC 9276). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9276

Internet Systems Consortium. (s.f.). *DNSSEC guide*. BIND 9 Administrator Reference Manual. https://bind9.readthedocs.io/en/latest/dnssec-guide.html

OpenBSD Project. (2022). *OpenSSH 9.0 release notes*. OpenSSH. https://www.openssh.com/txt/release-9.0

OpenBSD Project. (s.f.). *sshd_config(5) — OpenSSH daemon configuration file*. OpenBSD manual pages. https://man.openbsd.org/sshd_config

Postel, J., & Reynolds, J. (1983). *Telnet protocol specification* (RFC 854). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc854

Requena Llorentty, J. N. (2026a). *Guía de laboratorio 09: DNS profesional: vistas (split-horizon), DNSSEC con dnssec-policy y monitoreo* [Guía de laboratorio]. Universidad Privada Domingo Savio.

Requena Llorentty, J. N. (2026b). *Guía de laboratorio 10: TELNET vs SSH: la prueba del espía, la identidad del servidor y el acceso con claves ed25519* [Guía de laboratorio]. Universidad Privada Domingo Savio.

Requena Llorentty, J. N. (2026c). *Guía de laboratorio 11: SSH de nivel profesional: endurecimiento, SFTP enjaulado, fail2ban y túneles* [Guía de laboratorio]. Universidad Privada Domingo Savio.

Security Magazine. (2021, 26 de abril). *Unsecured computer attacked 51 times per minute*. https://www.securitymagazine.com/articles/95082-unsecured-computer-attacked-51-times-per-minute

Wireshark Foundation. (s.f.). *Following protocol streams*. Wireshark User's Guide. https://www.wireshark.org/docs/wsug_html_chunked/ChAdvFollowStreamSection.html

Ylonen, T., & Lonvick, C. (Ed.). (2006). *The Secure Shell (SSH) transport layer protocol* (RFC 4253). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc4253